

COMPUTER VISION LABS 1–6

Complete Exam Notes — Concepts + Coding + Line-by-Line Understanding

Simple language • Python/OpenCV • Viva preparation • Coding cheat sheets

SOURCE / SCOPE NOTE

These notes are grounded in the supplied Computer Vision Labs 1–6 guide and the visible structure of the GitHub repository. The supplied guide gives detailed material for Labs 1–5. The repository shows a Lab 6 folder, but its actual contents were not available to read here. Therefore, Lab 6 is not fabricated. Lab 2's missing experiment is reconstructed from the supplied Geometric Transformations material, and Lab 4's missing repository code is filled from the supplied guide.

How to use these notes

1. First read the “What is it?” section of each lab.
2. Then understand the code template rather than memorizing every line blindly.
3. For the exam, memorize the small core code blocks under “Must Know Code”.
4. Use the viva questions at the end of each lab to test yourself.
5. If the examiner gives a new image, identify the task first, then choose the matching technique.

Master Roadmap

Lab	Main Topic	Core Things to Know
1	Basic Image Processing	Read/display, grayscale, resize, blur, crop, text, threshold, rotation, blending, histogram equalization
2	Geometric Transformations	Rotation, scaling, shear, translation, rigid transform, perspective/homography
3	Feature Extraction + Filtering	HOG, LBP, color histogram, HED/HIG, convolution, Sobel, Canny, LoG
4	Image Segmentation	Global/adaptive/Otsu/color thresholding, hysteresis, region growing, watershed, K-means, deep learning
5	Wavelets, Boundaries, Hough, SIFT	Wavelets, Sobel/Canny/LoG, line/circle Hough, SIFT pipeline
6	Repository-specific	Exact Lab 6 source/code was not accessible in the supplied material; see the marked section

Absolute-Zero Foundation

Computer Vision means making a computer process and understand images or video. An image is basically a grid of pixels. Each pixel stores numerical information about brightness or color.

Term	Simple meaning
Pixel	Smallest unit/dot of an image
Resolution	Image dimensions, e.g. width × height
RGB	Red, Green, Blue color channels
Grayscale	One intensity channel: 0 is black, 255 is white
BGR	OpenCV's usual color order when reading an image
Feature	Useful visual information such as an edge, corner, texture, or keypoint

Segmentation	Splitting an image into meaningful regions/objects
Kernel	Small matrix moved over an image for filtering/convolution
Threshold	A value used to separate pixels into groups
Histogram	Counts/distribution of pixel values or feature values

LAB 01: Introduction to Computer Vision & Basic Image Operations

1. What is this lab about?

Lab 1 builds the basic skill of loading an image into Python, displaying it, changing its representation, changing its size, removing noise, selecting regions, adding annotations, and performing simple intensity operations.

2. Important libraries

Library	Why we use it
cv2 / OpenCV	Read, transform, filter, and process images
matplotlib.pyplot	Display images and plots
numpy	Work with image arrays and numerical operations
pandas	Analyze pixel values as a table

3. Read and display an image

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(image_rgb)
plt.title('My Image')
plt.axis('off')
plt.show()
```

Line-by-line

- `'import cv2'` loads OpenCV.
- `'import matplotlib.pyplot as plt'` gives us plotting/display functions.
- `'cv2.imread(...)'` reads the image into a NumPy array.
- OpenCV normally reads color images as BGR.
- `'cv2.cvtColor(..., COLOR_BGR2RGB)'` changes BGR to RGB for normal Matplotlib display.
- `'imshow'` displays the array; `'axis('off)'` hides axes; `'show()'` renders it.

VIVA TIP

If asked why BGR → RGB: OpenCV uses BGR ordering for its usual image reading, while Matplotlib expects RGB ordering.

4. Grayscale

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')
plt.show()
```

Grayscale removes color information and keeps intensity/brightness. It is commonly used before thresholding and edge detection.

5. Resize

```
resized = cv2.resize(image, None, fx=0.5, fy=0.5)
```

- `'fx=0.5'` means half the width.
- `'fy=0.5'` means half the height.
- Resizing is useful for faster processing or standardizing image size.

6. Gaussian Blur

```
blurred = cv2.GaussianBlur(image, (31, 31), 0)
```

- The `(31,31)` is the kernel size.
- Gaussian blur smooths the image and reduces noise.
- Blurring before edge detection helps reduce false/noisy edges.

7. Cropping / ROI

```
mid_x = image.shape[1] // 2
left = image[:, :mid_x]
right = image[:, mid_x:]
```

```
# General crop:
roi = image[100:300, 200:400]
```

ROI means Region of Interest. NumPy slicing is written as [rows, columns], which for images means [y, x].

8. Add text

```
cv2.putText(image, "Hello", (150, 150),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 0, 0), 2)
```

- `(150,150)` is the starting position.
- The font constant selects the font.
- `2` is font scale.
- `(255,0,0)` is a BGR color tuple in OpenCV.
- The last `2` is line thickness.

9. Pixel statistics with pandas

```
import pandas as pd
```

```
flattened = image.reshape(-1, 3)
df = pd.DataFrame(flattened, columns=['B', 'G', 'R'])
print(df.describe())
```

An image has 2D spatial structure plus color channels. Reshaping makes each pixel one row so pandas can calculate statistics such as mean, minimum, maximum, and quartiles.

10. Thresholding

```
_, thresh = cv2.threshold(gray, 100, 200, cv2.THRESH_BINARY)
```

- 100 is the threshold.
- Pixels above the threshold become the maximum value, here 200.
- Pixels at or below the threshold become 0.
- `\_` ignores the threshold value returned by OpenCV.

11. Rotation

```
height, width = image.shape[:2]
center = (width // 2, height // 2)
```

```
M = cv2.getRotationMatrix2D(center, 45, 1)
rotated = cv2.warpAffine(image, M, (width, height))
```

First we find the center. Then we create a 2×3 affine transformation matrix M. `warpAffine` applies it to the image.

12. Image blending

```
blended = cv2.add(image1, image2)
```

The images must have compatible sizes/channels. OpenCV adds corresponding pixel values and saturates at 255.

13. Histogram equalization

```
equalized = cv2.equalizeHist(gray)
```

It redistributes grayscale intensities to improve contrast, especially when an image has poor contrast.

Lab 1 — Must Know Code

```
import cv2, matplotlib.pyplot as plt

image = cv2.imread('image.jpg')
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blurred = cv2.GaussianBlur(gray, (5,5), 0)
resized = cv2.resize(image, None, fx=0.5, fy=0.5)
cropped = image[100:300, 200:400]

_, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
equalized = cv2.equalizeHist(gray)
```

Lab 1 Viva

- What is a pixel?
- What is resolution?
- Why convert BGR to RGB?
- Why use grayscale?
- What does Gaussian blur do?
- What is ROI?
- What does thresholding do?
- What does `cmap='gray'` do?
- Why is `\_` used in `\_ , thresh`?
- What is histogram equalization used for?

LAB 02: Geometric Transformations

MISSING EXPERIMENT — RECONSTRUCTED

The supplied material contains the complete Geometric Transformations topic and transformation codes but does not contain a separate formal experiment statement. For exam preparation, use the experiment below: implement and compare rotation, scaling, shearing, translation, rigid transformation, and perspective transformation on an input image.

1. What are geometric transformations?

They change where pixels are located. The goal is to move, rotate, resize, skew, or warp an image without changing the basic image content.

2. Coordinate system

- Image origin is normally at the top-left.
- x increases toward the right.
- y increases downward.
- A pixel can be addressed as `image[y, x]`.

3. Transformation hierarchy

Type	Main idea	Parallel lines?	Typical use
Linear	Rotation/scaling-like linear mapping around origin	Yes	Basic geometry
Affine	Linear transform + translation	Yes	Rotation, shear, translation
Projective	Perspective/homography	No	Document rectification, panorama

4. Rotation

```
center = (width // 2, height // 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(image, M, (width, height))
```

- `center` is the rotation center.
- `45` is the angle in degrees.
- `1.0` is the scale factor.
- `getRotationMatrix2D` creates a 2×3 matrix.
- `warpAffine` applies the affine transformation.

5. Scaling

```
scaled = cv2.resize(image, None, fx=3, fy=3,
                    interpolation=cv2.INTER_LINEAR)
```

This makes the image $3 \times$ larger in both dimensions. Interpolation estimates new pixel values.

6. Shearing

```
M = np.float32([
    [1, 0.5, 0],
    [0, 1, 0]
])
sheared = cv2.warpAffine(image, M, (new_width, height))
```

Shearing skews the image. Here x is changed using a term involving y, so vertical structures slant sideways.

7. Translation

```
M = np.float32([
    [1, 0, 100],
    [0, 1, 50]
])
translated = cv2.warpAffine(image, M, (width, height))
```

The image moves 100 pixels in x (right) and 50 pixels in y (down).

8. Rigid transformation

```
M = cv2.getRotationMatrix2D(center, 30, 1.0)
M[0, 2] += 100
M[1, 2] += 50

rigid = cv2.warpAffine(image, M, (width, height))
```

This combines rotation and translation while keeping scale and shear unchanged.

9. Perspective / Homography

```
src = np.float32([
    [0, 0],
    [w-1, 0],
    [w-1, h-1],
    [0, h-1]
])

dst = np.float32([
    [100, 50],
    [w-150, 0],
    [w-50, h-80],
    [50, h-20]
])

M = cv2.getPerspectiveTransform(src, dst)
warped = cv2.warpPerspective(image, M, (w, h))
```

- `src` contains the four original corner points.
- `dst` contains the four desired corner positions.
- `getPerspectiveTransform` calculates the homography matrix.
- `warpPerspective` performs the projective warp.

10. What to memorize

Task	Function
Rotate	getRotationMatrix2D + warpAffine
Scale	resize
Shear	custom 2×3 matrix + warpAffine
Translate	translation matrix + warpAffine
Rigid	rotation matrix + translation offsets
Perspective	getPerspectiveTransform + warpPerspective

Lab 2 Viva

- What is a geometric transformation?
- What is the difference between affine and projective transformation?
- Why is a 2×3 matrix used with warpAffine?
- What is translation?
- What is shearing?

- What is a homography?
- Why are four point pairs used for perspective transformation?
- Which transformation is useful for document scanning?
- What happens to parallel lines under perspective transformation?

LAB 03: Feature Extraction, Convolution & Edge Detection

1. What is feature extraction?

Feature extraction reduces a complex image into compact, useful information that can help recognition, classification, matching, or detection.

Feature type	Examples	Main purpose
Local	Keypoints, corners, patches	Matching and correspondence
Global	Color histograms, texture, moments	Classification/description

2. HOG — Histogram of Oriented Gradients

```
from skimage.feature import hog
```

```
features, hog_image = hog(  
    gray,  
    pixels_per_cell=(8,8),  
    cells_per_block=(2,2),  
    visualize=True  
)
```

- HOG focuses on local gradient directions.
- The image is divided into cells.
- Each cell contributes a gradient-direction histogram.
- Blocks are normalized.
- The final descriptor is a feature vector.
- Common use: object/pedestrian detection.

3. LBP — Local Binary Pattern

```
from skimage import feature
```

```
lbp = feature.local_binary_pattern(  
    gray, 8, 1, method='uniform'  
)
```

LBP compares neighboring pixels with a center pixel and creates a binary pattern. It is useful for texture description.

4. Color histogram

```
hist = cv2.calcHist(  
    [image], [0,1,2], None,  
    [8,8,8],  
    [0,256,0,256,0,256]  
)
```

A color histogram counts how frequently colors occur. More bins give finer color separation but can increase feature size.

5. HED / HIG

```
gradient_orientation = np.arctan2(gy, gx) * 180 / np.pi  
hist, _ = np.histogram(  
    gradient_orientation,  
    bins=8,  
    range=(0,360)  
)
```

- HED describes the distribution of edge directions.
- HIG describes the distribution of intensity gradients.

- Sobel gradients are commonly used to obtain gradient information.

6. Convolution

Convolution means sliding a small matrix called a kernel over an image, multiplying corresponding values, and combining them to produce a new value.

Filter	Purpose
Box blur	Simple smoothing
Gaussian	Smoother/noise reduction
Sobel	Gradient/edge detection
Emboss	Relief/3D-like visual effect

7. Sobel

```
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

```
magnitude = np.sqrt(gx**2 + gy**2)
```

- Gx measures change in the x direction.
- Gy measures change in the y direction.
- Magnitude combines both gradient components.
- Large magnitude usually means a strong intensity change/edge.

8. Canny

```
blurred = cv2.GaussianBlur(gray, (5,5), 1.4)
edges = cv2.Canny(blurred, 50, 150)
```

- Gaussian smoothing reduces noise.
- Gradient calculation finds intensity changes.
- Non-maximum suppression makes edges thin.
- Hysteresis thresholding connects strong and relevant weak edges.

9. LoG — Laplacian of Gaussian

```
blurred = cv2.GaussianBlur(gray, (5,5), 1.4)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
```

LoG first smooths and then applies the Laplacian. The supplied guide describes edges using zero-crossings and multi-scale analysis.

Lab 3 Viva

- What is a feature?
- Difference between local and global features?
- What does HOG capture?
- How does LBP represent texture?
- What is a kernel?
- What is convolution?
- Difference between Gx and Gy?
- What are the four main Canny steps?
- What is the purpose of Gaussian smoothing before Canny?

LAB 04: Image Segmentation

REPOSITORY CODE GAP

The visible repository structure did not show a Lab 4 folder, but the supplied guide contains the Lab 4 syllabus and code templates. The following section therefore uses the supplied guide to prepare the missing Lab 4 coding material.

1. What is segmentation?

Segmentation partitions an image into meaningful regions, usually to separate objects from the background or separate different objects.

2. Global thresholding

```
_, binary = cv2.threshold(
    gray, 128, 255, cv2.THRESH_BINARY
)
```

One threshold is applied to the entire image. It works best when foreground and background have clear intensity separation.

3. Adaptive thresholding

```
binary = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)
```

- Different image regions can get different thresholds.
- Useful when lighting changes across the image.
- `11` is the neighborhood/block size.
- `2` is the constant subtracted from the local statistic.

4. Otsu's thresholding

```
_, binary = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Otsu automatically selects a threshold from the image histogram. The supplied guide highlights bimodal histograms as the useful case.

5. Color segmentation using HSV

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(
    hsv,
    (30, 50, 50),
    (60, 255, 255)
)
```

- Convert BGR to HSV.
- Choose lower and upper HSV bounds.
- `inRange` creates a binary mask for pixels inside the range.

6. Hysteresis thresholding

```
edges = cv2.Canny(image, 100, 200)
```

Canny uses two thresholds: a high threshold for strong edges and a low threshold for weaker connected edges.

7. Region growing

Start from a seed pixel and grow the region by adding neighboring pixels that are sufficiently similar. The supplied guide gives this concept rather than a complete OpenCV implementation.

8. Watershed

```
cv2.watershed(image, markers)
```

Watershed treats the image like a topographic surface and uses markers as starting regions. It is especially useful for separating touching/overlapping objects.

9. K-Means segmentation

```
pixel_values = image.reshape((-1, 3)).astype(np.float32)
```

```
criteria = (  
    cv2.TERM_CRITERIA_EPS +  
    cv2.TERM_CRITERIA_MAX_ITER,  
    100,  
    0.2  
)
```

```
K = 4
```

```
_, labels, centers = cv2.kmeans(  
    pixel_values,  
    K,  
    None,  
    criteria,  
    10,  
    cv2.KMEANS_RANDOM_CENTERS  
)
```

- Reshape the image so each pixel becomes a 3-value data point.
- K is the number of desired clusters.
- K-means groups pixels according to similarity.
- Centers represent the cluster colors.

10. Deep-learning segmentation

- U-Net
- Mask R-CNN
- DeepLab
- These are suitable for more complex segmentation tasks when appropriate data and training are available.

11. Active contours / snakes

An active contour is a deformable model that evolves toward object boundaries. The supplied guide lists tracking and boundary refinement as uses.

Lab 4 — Technique selection

Situation	Technique
Clear foreground/background intensity	Global threshold
Uneven lighting	Adaptive threshold
Automatic threshold selection	Otsu
Known object color	HSV + inRange
Touching objects	Watershed
Color-based grouping with multiple regions	K-Means
Complex learned segmentation	U-Net / Mask R-CNN / DeepLab

Lab 4 Viva

- What is segmentation?
- Global vs adaptive thresholding?
- What does Otsu do automatically?
- Why use HSV for color segmentation?
- What is a mask?
- Why is watershed useful for touching objects?
- What is K in K-means?
- What is a marker in watershed?
- Name deep-learning segmentation models.

LAB 05: Wavelets, Boundary Detection, Hough Transform & SIFT

1. Wavelet transformation

Wavelets analyze image information at different frequencies/scales.

Type	Simple idea	Use from supplied guide
CWT	Continuous analysis	Seismic/oscillatory signals
DWT	Discrete decomposition	Compression/denoising
WPT	Flexible decomposition	Complex frequency analysis
Inverse	Reconstruction	Image reconstruction/compression

2. Boundary detection with Sobel

```
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = np.sqrt(gx**2 + gy**2)
```

Sobel detects intensity changes. The gradient magnitude indicates how strong the local change is.

3. Boundary detection with Canny

```
blurred = cv2.GaussianBlur(gray, (5,5), 1.4)
edges = cv2.Canny(blurred, 50, 150)
```

The guide emphasizes Canny for accurate, thin and continuous edges.

4. LoG

```
blurred = cv2.GaussianBlur(gray, (5,5), 1.4)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
```

5. Hough Transform — lines

The basic line equation is $y = mx + b$. Hough transform converts line detection into a voting problem: image evidence votes for possible lines, and peaks represent likely lines.

```
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi/180,
    100,
    minLineLength=50,
    maxLineGap=10
)
```

- `1` is the distance resolution in pixels.
- `np.pi/180` is the angle resolution in radians.
- `100` is the vote threshold.
- `minLineLength` rejects very short lines.
- `maxLineGap` controls how far apart segments can be and still be joined.

6. Hough Transform — circles

```
circles = cv2.HoughCircles(  
    gray,  
    cv2.HOUGH_GRADIENT,  
    1,  
    20,  
    param1=50,  
    param2=30,  
    minRadius=0,  
    maxRadius=0  
)
```

A circle can be described by $(x-a)^2 + (y-b)^2 = r^2$. The transform searches for likely center/radius combinations.

7. SIFT

SIFT (Scale-Invariant Feature Transform) extracts local features designed to be robust to scale and rotation changes.

6. Scale-space extrema — Difference of Gaussians (DoG) finds candidate keypoints.
7. Keypoint localization — unstable/low-quality candidates are rejected.
8. Orientation assignment — dominant gradient direction is assigned.
9. Descriptor generation — a 128-value descriptor is formed using the $4 \times 4 \times 8$ structure described in the guide.
10. Matching + RANSAC — descriptors can be compared and outliers rejected.

```
sift = cv2.SIFT_create()  
keypoints, descriptors = sift.detectAndCompute(  
    gray, None  
)
```

- `keypoints` are detected interesting locations.
- `descriptors` are numerical descriptions of those locations.
- SIFT is useful for image stitching, object recognition, and tracking.

Lab 5 Viva

- What is a wavelet?
- What is the difference between CWT and DWT?
- What is Hough transform?
- What is the equation of a line?
- What is the equation of a circle?
- What does HoughLinesP return?
- What is SIFT?
- Why is SIFT useful for matching?
- What is a keypoint?
- What is a descriptor?
- What is RANSAC used for?

LAB 06: Repository-Specific Lab — Exact Material Required

IMPORTANT — DO NOT MEMORIZE INVENTED LAB 6 CONTENT

The visible GitHub repository structure shows a Lab 6 folder, but the actual files/code inside that folder were not available through the accessible source. The supplied guide also has a summary labelled “All 6 Labs” but only provides detailed sections for Labs 1–5. Because the exact Lab 6 experiment is not supported by the available source, this document intentionally does not pretend a guessed topic is your Lab 6.

What you should do before the exam

11. Open the Lab 6 folder in your GitHub repository.
12. If it contains a notebook (.ipynb), Python file (.py), or PDF/Word experiment, upload it here.
13. Once the exact file is available, its code can be added to this document with line-by-line explanations.
14. Until then, focus strongly on Labs 1–5 because those are the topics explicitly supported by the supplied guide.

Integrated coding patterns you should still recognize

```
# Read
image = cv2.imread('image.jpg')

# Convert
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Basic processing
blurred = cv2.GaussianBlur(gray, (5,5), 0)
resized = cv2.resize(image, None, fx=0.5, fy=0.5)
cropped = image[100:300, 200:400]

# Threshold
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Edges
edges = cv2.Canny(blurred, 50, 150)

# Features
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)

# Hough
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 100,
                        minLineLength=50, maxLineGap=10)
```


MASTER EXAM CHEAT SHEET

1. If the examiner asks “Which technique should I use?”

Problem	Technique / Code
Load/show image	cv2.imread + matplotlib
Convert to grayscale	cv2.cvtColor(..., COLOR_BGR2GRAY)
Reduce noise	cv2.GaussianBlur
Resize	cv2.resize
Crop region	NumPy slicing
Binary image	cv2.threshold
Automatic threshold	THRESH_OTSU
Uneven lighting	adaptiveThreshold
Segment by color	HSV + inRange
Fast/simple edges	Sobel
Thin edge map	Canny
Second-derivative edge analysis	Laplacian / LoG
Rotate	getRotationMatrix2D + warpAffine
Move image	translation matrix + warpAffine
Skew image	shear matrix + warpAffine
Perspective/document warp	getPerspectiveTransform + warpPerspective
Texture	LBP
Gradient-based descriptor	HOG
Detect lines	HoughLinesP
Detect circles	HoughCircles
Feature matching	SIFT
Touching objects	Watershed
Cluster colors	K-Means

2. Most important function signatures

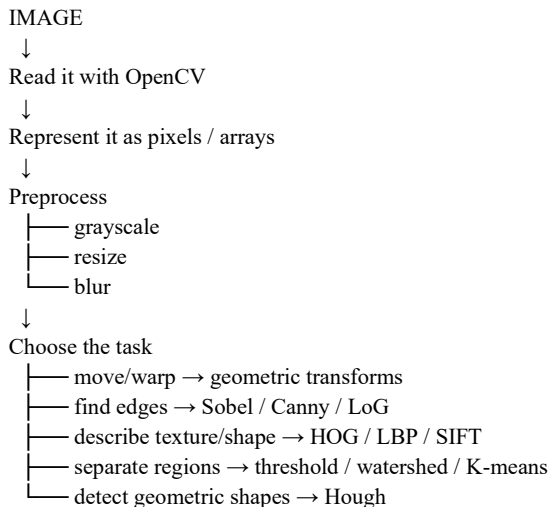
```
cv2.imread(path)
cv2.cvtColor(image, code)
cv2.resize(image, dsize_or_None, fx=..., fy=...)
cv2.GaussianBlur(image, (ksize, ksize), sigma)
cv2.threshold(gray, threshold, maxValue, type)
cv2.adaptiveThreshold(gray, maxValue, adaptiveMethod, thresholdType, blockSize, C)
cv2.Sobel(gray, ddepth, dx, dy, ksize=3)
cv2.Canny(image, threshold1, threshold2)
cv2.Laplacian(image, ddepth)
cv2.getRotationMatrix2D(center, angle, scale)
cv2.warpAffine(image, M, (width, height))
cv2.getPerspectiveTransform(src, dst)
cv2.warpPerspective(image, M, (width, height))
cv2.HoughLinesP(...)
cv2.HoughCircles(...)
cv2.SIFT_create()
sift.detectAndCompute(gray, None)
```

3. Common mistakes in coding exams

- Forgetting to import cv2, numpy, matplotlib, or the required skimage function.
- Using a color image where a grayscale image is expected by an operation.
- Forgetting BGR → RGB when displaying OpenCV images with Matplotlib.
- Mixing up image[y, x] with image[x, y].
- Using an invalid Gaussian kernel size; kernel dimensions should be appropriate/odd for the standard OpenCV use.
- Using variables such as w or h before defining them.

- For perspective transformation, giving fewer/mismatched source and destination points.
- For K-means, forgetting to reshape pixels into rows.
- For SIFT, confusing keypoints with descriptors.
- For Hough detection, expecting perfect results without suitable edge preprocessing.

4. One-minute mental model



5. Final Viva Rapid-Fire

Question	Short answer
What is computer vision?	Processing/understanding visual data such as images and video.
What is a pixel?	The smallest image element containing intensity/color information.
Why grayscale?	It simplifies an image to intensity values and is useful for many processing tasks.
Why Gaussian blur?	To smooth the image and reduce noise.
What is thresholding?	Separating pixels into groups using an intensity threshold.
What is segmentation?	Partitioning an image into meaningful regions.
What is convolution?	Sliding a kernel over an image and combining local values.
What does Sobel find?	Intensity gradients/edges.
Why Canny?	It provides a structured multi-stage edge detection process.
What is HOG?	A descriptor based on local gradient-direction histograms.
What is LBP?	A texture descriptor based on comparing neighboring pixels with a center pixel.
What is Hough transform?	A voting-based method for detecting parameterized geometric shapes.
What is SIFT?	A scale- and rotation-invariant local feature detector/descriptor.
What is homography?	A projective mapping between planes used for perspective warping.
Why watershed?	To separate regions, especially touching objects.
What is K in K-means?	The number of clusters to create.

Final note

These notes are designed to make the coding understandable from zero rather than only giving memorization material. For the exact Lab 6 content, the actual repository file is still needed; once supplied, it can be inserted into the same format.